Mathematical foundations & Derivation for Recurrent Neural Network (RNN)

Objective : To understand how an RNN processes seq^n data, maintain information across time step's, & learn through backpropagation through time (BPTT)

Example:

If a sentence says:

"The food was not good"

If some model see "good" it might say it is a positive comment, But "not" changes the whole sentence.

So, We use seq^n model's for such problem's because in these types of problems sequence of the data matters.

Why RNN? (& what it does)

unlike a MLP, it adds a mechanism for carrying information from one time step to the next.

we call it hidden state h_t.

Current I/p + Previous state ⟶ New state

Mathematical Notation & Equations:

$$X_{1,t} = (x_1, x_2, x_3, \ldots, x_t)$$

$x_{1,t}$ = the input seq^n.

t = total number of time stamps.

X_t = i/p at time stamp t.

As, discussed above we can say that

current hidden State is a function of current I/P &, previous hidden state (which carries info of word processed earlier)

$$h_t = f(X_t, h_{t-1})$$

X_t = Current I/P

h_t = new hidden State

h_{t-1} = previous hidden State

So, we would I/P the h_{t-1} & X_t to a Single Neuron to Combine the information from both, by trainable weighted Combination.

a neuron computes

$$Z = wx + b$$

but as info sources are two so,

$$a_t = W_{xh} X_t + W_{hh} h_{t-1} + b_h$$

now we would need an activation function; to capture non-line-arity of data

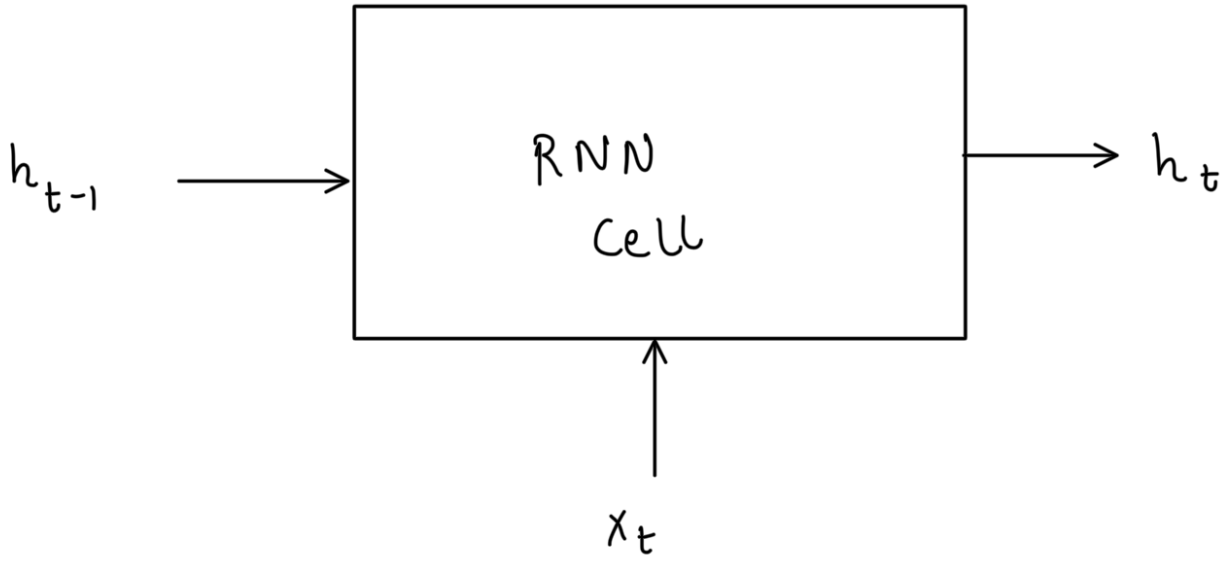
for this we choose tanh activation func^n.

$$h_t = \tanh(a_t)$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

finally we get:

$$h_t = \tanh(W_{xh} X_t + W_{hh} h_{t-1} + b_h)$$



Let's assume

d_x : number of Input features. (3 features)

d_h: number of hidden units. (4 hidden units)

↳ this is number of neurons in the layer

Input vector	$x_t \in \mathbb{R}^{3\times 1}$
Input weight	$W_{xh} \in \mathbb{R}^{4\times 3}$
Recurrent weight	$W_{hh} \in \mathbb{R}^{4\times 4}$
hidden state	$h_t \in \mathbb{R}^{4\times 1}$

so, we take hidden state and apply another trainable linear transformation.

$$o_t = W_{hy} h_t + b_y$$

o_t: output

b_y: output bias

W_{hy}: hidden to o/p weight matrix

h_t: hidden state

now, we convert o_t i.e o/p score to probability by using Softmax function

So, we get

$$\hat{y}_{t,i} = e^{o_{t,i}}$$

$$\frac{}{\sum_j e^{o_{t,j}}}$$

d_y: output class dimension = 2 (+ve & -ve sentiment)

for exp: if $o_t = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

Class 1 is negative & class 2 is positive

Calculate the +ve probability

$$\hat{y}_{t,2} = \frac{e^2}{\sum_{j=1}^{2} e^{o_{t,j}}} = \frac{e^2}{e^1 + e^2} = 0.7321$$

$$\hat{y}_{t,1} = \frac{e^1}{e^1 + e^2} = 0.269$$

Loss function (cross entropy loss)

$$L = -\sum_{i=1}^{d_y} y_i \log(\hat{y}_i)$$

y_i = actual label +ve or -ve

$\hat{y}_i$ = predicted label

d_y = number of classes

L = Loss

$y_{neg} = 0, \quad y_{pos} = 1$

for exp corresponding to previous eqn:

$$L = -[0 \log(0.2) + 1 \log(0.8)]$$

$\boxed{L = 0.223}$ this the loss of it

How Loss connect to our RNN?

$x_{i,t} \longrightarrow h_t \longrightarrow \hat{y} \longrightarrow L$

for our forward pass with produce predictions & then calculate the Loss for it.

Unrolling through time

If we take 3 word sentences, calculation would be carried assuming it.

$h_0 = 0$ (initial hidden state)

$i = 1, 2, 3$

$$= \tanh\left(W_{xh} X_i + W_{hh} h_{i-1} + b_h\right)$$

BPTT

forward pass:

$$x_1 \longrightarrow h_1 \longrightarrow h_2 \longrightarrow h_3 \longrightarrow \hat{y} \longrightarrow L$$

Loss is calculated at the end, but h_i $(i = 1,2,3)$ helped produce that loss, but our target is to minimize the loss.

So, we trace the error backwards to minize the Loss, this is called <u>Backpropagation</u>.

Backpropagation is just a big Chain rule.

$\therefore$

$$\frac{\partial h}{\partial h_{t-1}} = \frac{\partial h}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}}$$

$$\frac{d}{da} \tanh(a) = 1 - \tanh^2(a)$$

as $h_t = \tanh(a_t)$

$$\frac{\partial h_t}{\partial a_t} = 1 - h_t^2 \qquad \text{——— (1)}$$

$$a_t = W_{xh} x_t + W_{hh} h_{t-1} + b_h$$

$$\frac{\partial a_t}{\partial h_{t-1}} = W_{hh} \qquad \text{——— (2)}$$

multiply (1) & (2)

we get:

$$\frac{\partial h_t}{\partial h_{t-1}} = (1 - h_t^2) W_{hh}$$

(*) but $(1 - h_t^2)$ is a (3×1) vector, so to convert it to an matrix

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h_t^2) W_{hh} \qquad \text{——— (3)}$$

So, finally we get

$$\boxed{\frac{\partial L}{\partial h_{t-1}} = W_{hh}^T \, \text{diag}(1 - h_t^2) \frac{\partial L}{\partial h_t}}$$

Starting with the o/p equation

$$o_t = W_{hy} h_t + b_y$$

$$\hat{y}_t = \text{softmax}(o_t)$$

Loss at step t is:

$$L_t = -\sum_i y_{t,i} \log \hat{y}_{t,i}$$

$$L_t = -\sum y_{t,i} \log \left(\frac{e^{o_{t,i}}}{\sum_i e^{o_{t,i}}} \right)$$

$$L_t = -\sum y_{t,i} o_{t,i} + \log \sum_i e^{o_{t,i}}$$

$$\boxed{\frac{\partial L_t}{\partial o_t} = \hat{y}_t - y_t}$$

Recall the o/p eqn

$$o_t = W_{hy} h_t + b_y$$

$$\frac{\partial L_t}{\partial W_{hy}} = \frac{\partial L_t}{\partial o_t} \frac{\partial o_t}{\partial W_{hy}} = (\hat{y}_t - y_t) h_t^T$$

dimension of matrix is $(d_y \times 1)(1 \times d_h) = d_y \times d_h$

So, we can also tell the total loss as:

$$L = \sum_{t=1}^{T} L_t$$

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^{T} (\hat{y}_t - y_t) h_t^T$$

similarly, we can get

$$\frac{\partial L}{\partial b_y} = \sum_{t=1}^{T} (\hat{y}_t - y_t)$$

now, we want to know how Loss change with h_t

$$\frac{\partial L_t}{\partial h_t} = \left(\frac{\partial o_t}{\partial h_t}\right)^T \left(\frac{\partial L_T}{\partial o_t}\right)$$

substitution gives

$$\frac{\partial L_t}{\partial h_t} = W_{hy}^T (\hat{y}_t - y_t)$$

h_t affect 2 paths current & future / next

so,

$$\frac{\partial L}{\partial h_t} = W_{hy}^T (\hat{y}_T - y_T) + W_{hh}^T \text{diag}(1 - h_{t+1}^2) \frac{\partial L}{\partial h_{t+1}}$$

gradient for hidden bias:

$$\frac{\partial L}{} = \frac{\partial L}{} \frac{\partial a_t}{} = \overbrace{\left[\frac{\partial L}{} \cdot (1 - h_t^2)\right]}^{\delta(t)} \left[I \right]$$

$$\frac{\partial L}{\partial b_h} \quad \frac{\partial h_t}{\partial a_t} \quad \frac{\partial a_t}{\partial b_h}$$

$$\frac{\partial L}{\partial b_h} = \sum_{t=1}^{T} \delta_t$$

W_{xh} gradient calculation

$$\frac{\partial a_{t,i}}{\partial W_{x_t h, ij}} = x_{t,j}$$

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^{T} \delta_t \, x_t^T$$

W_{hh} gradient calculation

Similar to previous weight

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \delta_t \, h_{t-1}^T$$

(*) **Summary All 5 parameter grad**

Param	Grad
W_{hy}	$\sum_t (\hat{y}_t - y_t) \, h_t^T$
b_y	$\sum_t (\hat{y}_t - y_t)$
W_{xh}	$\sum_t \delta_t \, x_t^T$

W_{xh} $\quad$ $\sum_t \delta_t x_t^T$

W_{hh} $\quad$ $\sum_t \delta_t h_{t-1}^T$

b_h $\quad$ $\sum_t \delta_t$

We have calculated the gradients, Now we have to use them to improve the RNN

Learning rate

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b}$$

we can use our weight & biases to substitute in the above 2 eqn.

update the parameter's & then give other sentence & Run the same step's & do it untill whole training is done.